BIOGOALS.SELECT_USER_MANUAL

User manual version: 1.0
Davide Carecci, 2025
TO DO: check on UIT machine the folders.

Indice

Preliminaries

The following user manual is intended to guide the other end-users of the *Biogoals.Select* tool. This tool is intended to be used by biomethane plant operators. It has the aim to provide a relatively simple/'conventional' way to track (*online*) a desired biomethane production value, adjusting the feeding of one co-feedstock and rejecting external disturbances while guaranteeing safety throughout the operations. It is based on feedback control theory: it employs two standard PI controllers in 'parallel' configuration, and extended for override capabilities of the integrator state when *switching* from one to the other.
When coupled with the setpoints derived from a diet optimisation carried out with the aid of the *Biogoals.agri-AcoDM* tool, it is a promising solution to avoid digesters' overly conservative working conditions i.e. pushing the loading condition up to its user-defined safety limit, increasing biomethane production.
The tool was experimentally proved (twice) able to stir *online* the biomethane production from a *nominal*/*initial* equilibrium/operating point to a *optimal/final* operating point.
It is compatible with nowadays full-scale applicability as it requires only the biogas total flow rate and volumetric composition in terms of CH_4 and CO_2 as *online*-measurable data.
It is composed by a methane flow rate ($q_{M,gb}$) PI controller called *header* and a CO_2 over CH_4 biogas composition ratio ($x_{C,gb}/x_{M,gb}$ i.e. 'safety' proxy) PI controller called *follower*.
When the 'safety' proxy is above a user-defined threshold, the main goal of the controller becomes the regulation of the 'safety' proxy itself (*follower* is active), whereas the biomethane production improvement is the primary goal when safe operations are in place

(*header*). The outputs' setpoints y_{ref} and the optimised feeding schedule of the other co-feedstocks d_{ref} are obtained with an *offline* agri-AcoDM optimisation done before the actual *online* transition from *initial* to *optimal* operations.
In the example of the experimental validations carried out during the PhD project, the control action u was the daily load of tomato sauce converted to an *on/off* PWM duty cycle of the actuator i.e. peristaltic pump. See IEEE_CDC2024 for further details.

To better understand the controllers and program behaviours, before actual *in plant* operation, the user can:

- run an *'open-loop'* example (just to check what the code does), running **'test_closedloop.ipynb'** commenting the 'integration' part;
- run an *'open-loop'* example (just to check what the code does), running **"UIT_selectorPI_operative.py"** commenting cells 13-14 and adding to the proper directory the data extracted from the UIT machine;
- run a 'closed-loop' example running **'test_closedloop.ipynb'** without commenting the 'integration' part. Note: a proper *Biogoals.agri-AcoDM* implementation is required in this case to serve as the *real plant* simulator. Some functions of the *Biogoals.Twin* library are also needed. See the 'Closed-loop test' section for further details;
- run a 'closed-loop' example running the bla bla model inside *Biogoals.agri-AcoDM*;
- run an 'offline' optimisation to find the best design of the selectorPI controller, running **"Optimization_selectorPI_design.ipynb"** a proper *Biogoals.agri-AcoDM* implementation is required in this case to serve as the *real plant* simulator.

Note: in addition to the standard Python libraries present in **requirements.txt** the custom Python library **general_utils** is needed to run these codes. How to install it:

- if the user prefers a local installation in his/her computer: download the repository from GitHub, then run `pip install "path_to\general_utils"` in your Python virtual environment/Jupyter kernel;
- otherwise, install directly from GitHub running `pip install git+https://github.com/DaveCacci/general_utils.git` or `pip install git+https://<GITHUB_TOKEN>@github.com/DaveCacci/general_utils.git` if the repository is private (ask the creator for the token if not already available or deprecated). Alternatively, just add `git+https://github.com/DaveCacci/general_utils.git` to **requirements.txt** and install everything at once `pip install -r requirements.txt`.

Files description

- ***"SelectorPI_pseudo_code.pdf"***: pseudo-code in PDF file as further material to understand the theory and the actual implementation of the controller.
- ***"SelectorPI_controller.py"***: Python file that contains the actual *'PIcontroller'* and *'HysteresisComparator'* classes (the two main blocks shown in the '*Closed-loop scheme*' section of this manual). Both the version implemented on the UIT reactor (i.e. old,

Cremona experimentation 2023-2024) and the one on the BioTA lab reactors (Chile experimentation 2025) are present inside.

- ***"Optimization_selectorPI_design.ipynb"***: Jupyter notebook to run an optimisation for the selectorPI design parameters (see the 'Design optimization' section of this manual for further details).
- ***"UIT_implementation/UIT_software_user_manual.pdf"***: user manual of the proprietary software of the UIT test facility.
- (MAIN) ***"UIT_implementation/UIT_selectorPI_operative.py"***: main controller code, program, Python file.
- ***"UIT_implementation/UIT_results_reader.ipynb"***: Jupyter notebook to read, analyse and plot the outputs of "*/UIT_selectorPI_operative.py*".
- ***"UITimplementation/setpoint{date}.csv"***: CSV input data file containing the setpoints.
- ***"UIT_implementation/Bioreactor/"***: sub-folder that contains the configuration INI files read by the UIT proprietary software on the UIT machine.
- ***"UIT_implementation/Figures"***: sub-folder to store figures created by the main control program.
- ***"UIT_implementation/Output"***: sub-folder to store the output CSV files created by the main control program.
- (MAIN) ***"RPi_implementation/RPi_selectorPI_operative.py"***: main controller code, program, Python file.
- ***"RPi_implementation/test_closedloop.ipynb"***: Jupyter notebook to run the main control program from. See the 'Closed-loop test' section for its description.
- ***"RPi_implementation/test_closedloop_integrator.ipynb"***: Jupyter notebook to run the Modelica integrator that mimics a real plant. To be coupled with the main control program inside *"test_closedloop.ipynb"* to simulate a closed-loop experiment. See the 'Closed-loop test' section for its description.
- ***"RPi_implementation/Test_closedloop"***: sub-folder to store an example of all the inputs/outputs needed for and generated by a complete closed-loop simulation with *"test_closedloop.ipynb"*, for the sake of the repository's readability. See the 'Closed-loop test' section for its description.
- ***"RPi_implementation/Plots"***: sub-folder to store the plots created by the main control program.
- ***"RPi_implementation/logs"***: sub-folder that saves a copy of the logging massages shown at runtime in console, for later inspection. Note: saves all messages returned by the *logging* Python-library, not the simpler messages returned using the *print* Python-keyword.
- ***"RPi_implementation/Input/y_df_data_on_alt_R1.csv"***: CSV input data file containing the *online* measurements already preprocessed and ready to be read for the computation of control action purposes.
- ***"RPi_implementation/Input/Output_setpoints.txt"***: TXT input data file containing the setpoints. Same for ".csv".

- ***"RPi_implementation/Input/SELECTOR_pwm_actual.csv"***: CSV file storing the control action values in terms of parameters of the duty cycle (to be read by the actuator). Note: it is stored inside 'Input' because it is physically an input to the actual system to be controlled, but it is actually an 'Output' of the program.
- ***"RPi_implementation/Output/Boundary_calc_conditions.csv"***: CSV output file containing measured data and setpoints' values at each control step.
- ***"RPiimplementation/Output/State{header/follower}_{date}.csv"***: CSV output file storing the quantities computed and required to run the *header* and *follower* PI controllers (e.g. activeness boolean condition, past integral terms, feedback errors etc.).
- ***"RPiimplementation/Output/Output_selector{date}.csv"***: CSV output file storing the feedback errors and actual control actions computed by the 'selectorPI' at each control step.

The program functioning is based on the use of the *pandas* Python library together with *Timestamps* i.e. objects created with the *datetime* library that index some other quantities with date and time values (e.g. 2025-10-24 13:25:00).
When implemented on real reactors (not in simulation with Biogoals.agri-AcoDM as real plant) the main difference between the "**RPi_selectorPI_operative.py**" and the "**UIT_selectorPI_operative.py**" is that the former reads already pre-processed data (by **"preprocess_meas.py"**, see note in the 'Closed-loop architecture' section of this manual), whereas data pre-processing is performed inside "**UIT_selectorPI_operative.py**" (it was the only Python file needed to run the controller on the UIT reactor). As a result, by now, "**RPi_selectorPI_operative.py**" can be run only at times equal to rounded hours (e.g. HH:00:00, not at HH:MM:00 with MM $\neq$ 00), whereas "**UIT_selectorPI_operative.py**" can be run at any time.

SelectorPI_controller.py

This code is a **collection of declarations of Python classes** and their methods (i.e. functions inside classes), so it is meant to be imported in other codes/notebooks, not to be run by its own. Some standard libraries are imported. The main difference between the classes used during the experimentation in Chile with RPi and the *'_uit'* classes is that the former were defined with the help of the **@dataclass** standard Python library and has a *logger* attribute that is an instantiation of the **logging** standard Python library (a better way to log intermediate info, warnings and errors when running codes online like in the case of a closed-loop controller). Since the main methods are practically the same or does the same operations, only the RPi formulation is going to be discussed here more in details (however, the code is overall well commented).

- The **PIcontroller** class: if a **Pcontroller** only is desired, as it was for the experimental validations, the user must COMMENT MANUALLY THE INTEGRAL ACTION AT LINE 57 and 236. It contains the functions to compute a new control action, load the past integral state, override the integral state with another value (e.g. the one of the other parallel

controller in the 'selector' scheme) and save out to a *pandas dataframe* the updated computed quantities.

- The **HysteresisComparator** class: it compares the value of the 'safety' proxy with user-defined thresholds (*threshold_low* < *threshold_high* to avoid chattering behaviours of the controller). It has a boolean state (True or False) that is changed if the input value to this function is above the *threshold_high* or below the the *threshold_low* (remains the same if within the two). Actually, it was modified to have change of state also if I go below *threshold_high*...not a problem when there are discrete data of biogas analysis...but in theory to be corrected!! .

"UIT_selectorPI_operative.py"

Standard libraries are imported as well as some custom functions present in a different or the same directory and, as mentioned above, some functions from the **general_utils** library. If a function present in another directory is needed, add `sys.path.insert(0, 'your_directory')` (e.g. 'SelectorPI_controller.py' in the example provided). Note that on the UIT machine present in the "*Fabbrica della Bioenergia*" A. Rozzi laboratory (Politecnico di Milano, Cremona campus), Windows 7 is installed, so that Python 3.8 is the last version supported (there are no major compatibility issues for the present code to run with such interpreter, but for the Pandas "*append*" commands that must be replaced with "*_append*"). See Chapter 6 of '*UIT_software_user_manual.pdf*' for further information on where to place this file to be correctly integrated with the UIT machine.

- The data from the UIT machine are saved in three different sub-folders ('*R1*', '*R2*' and '*Analyse*'). The data in, for example, the '*R1*' sub-folder are stored in CSV files with the date as filename in the format *'YYYY-MM-DD_R1.csv'*. The user SPECIFIES THE REACTOR NAME, THE DIRECTORY AND THE AMOUNTS OF DAYS OF INPUT DATA FILES TO BE READ. Two days (*today* and *yesterday*) are read in the example (minimum recommended) and concatenated in one unique Pandas *dataframe*.
- **Data preprocessing**: the user SETS THE THRESHOLD for outlier removal as an the maximum distance the Z-score of a data point can be from 0 in terms of integer-times the standard deviation of data. Data points characterised by a Z-score > *threshold* are substituted with a linear interpolation between the previous and the next 'non-outlier' data points (done on both biogas flow rate and CH_4 and CO_2 fractions). CH_4 flow rate and biogas CO_2/CH_4 ratio are computed. The user SETS THE WINDOW SIZE for the moving average of flow rates (in particular needed if the reactor mixing is carried out with on-off duty cycles). Data are extracted at the current control step *Timestamp* (last row of data).
- **Plot** the data (original signal vs outlier removed vs filtered with moving average).
- The user SPECIFIES THE DIRECTORY OF THE INPUT DATA FILE that contains the setpoints (Time or *Timestamp*, methane flow rate, biogas composition ratio and relative lower and higher thresholds). It can be in TXT or CSV format depending on how it is created. If CSV, as in the example, no need for further manipulation is needed. For

logging purposes, the user SPECIFIES THE DIRECTORY FOR AN OUTPUT CSV FILE where all the quantities of interest for the current control step are saved to.

- The user SETS THE CONTROL DESIGN PARAMETERS (control interval *dt* in seconds, *saturation_high* and *saturation_low* in $m^3\ s^{-1}$, *kp*, *Ti* in seconds). The controllers are instantiated with the **PIcontroller** class. The last-available states of the controllers' integrator part are loaded with the aid of the ***load_state*** method (the user SPECIFIES THE FILENAMES AND DIRECTORY FOR AN OUTPUT CSV FILE for both controllers).
- **Override**: extract the previous boolean conditions of *selection* for both controllers (which was the "active" one between the two). Update the *selection* based on the **HysteresisComparator** class (that compares the measured biogas composition ratio with thresholds) and its ***update*** method. If *selection* has changed its value at the current run, compute the value needed to override the integrator of the controller that is turning from inactive to active/selected to be consistent with the the last-available *control_signal*. Then, override with the aid of the ***reset_state*** method.
- **Compute** the control action for the current control step of both controllers and select the one of the controller that is active/selected for the current control step (if the measured biogas composition ratio is higher then the user-defined safety threshold, select the *control_signal* of the *'follower'* controller). Save the updated controllers' states with the aid of the ***save_state*** method. Note that the output of the ***compute*** method is in $m^3\ s^{-1}$ and it must be converted to mL d^{-1}.
- **Convert** the control action from a continuous *float* to a PWM on-off duty cycle couple (*on_minutes_ini, off_minutes_ini*). To do so, the on-time seconds of one period are fixed to 60 seconds (1 minute). Add the nominal/initial value of the control action (i.e. *on_minutes_nominal*) to the *control_signal* to obtain the actual value of the control action to be communicated to the actuation system (see Section *Additional notes* for details). The user SPECIFIES THE CHARACTERISTIC FLOW RATE OF THE PUMP. The user SPECIFIES THE FILENAME AND DIRECTORY FOR AN OUTPUT CSV FILE where all the quantities of interest computed in the current control step are saved to.
- The latest-available **configuration INI file** of the UIT proprietary software is loaded and **updated** with the above-mentioned PWM values (*on_seconds_ini, off_seconds_ini*). The user SPECIFIES INPUT, OUTPUT AND COPY PATHS OF FHE CONFIGURATON FILES.
- The user SPECIFIES THE DIRECTORY TO WRITR LOG with all success, warning and error messages.

"RPi_selectorPI_operative.py"

Standard libraries are imported as well as some custom functions present in a different or the same directory and, as mentioned above, some functions from the **general_utils** library. If a function present in another directory is needed, add `sys.path.insert(0, 'your_directory')` (e.g. 'SelectorPI_controller.py' in the example provided). Afterwards, the **main()** function is declared to allow the script to be executed from the terminal (with command-line arguments) and other scripts (nominally, this code is imported into

"test_closedloop.ipynb" to be run. The advice when performing simulations is too have both **"RPi_selectorPI_operative.py"** and **"test_closedloop.ipynb"** open in your editor, to modify this code and then testing it. If the user wants to run the present code as stand alone, delete **main()** function definition line and unindent the code below). For the reading and saving process of different simulation scenario avoiding the override of old results and confusion, the user SETS A *modelname* string to be used as **main()** argument. Is a specific folder is desired for the input and output files of the current simulation, SET A *testname* equal to the name of the desired subfolder to the specified *directory* (where this code is placed is default). The **logger** of the info (very comfortable and smart when running this code online) is setup SETTING LOGS SUBFOLDER AND FILE NAMES. Afterwards:

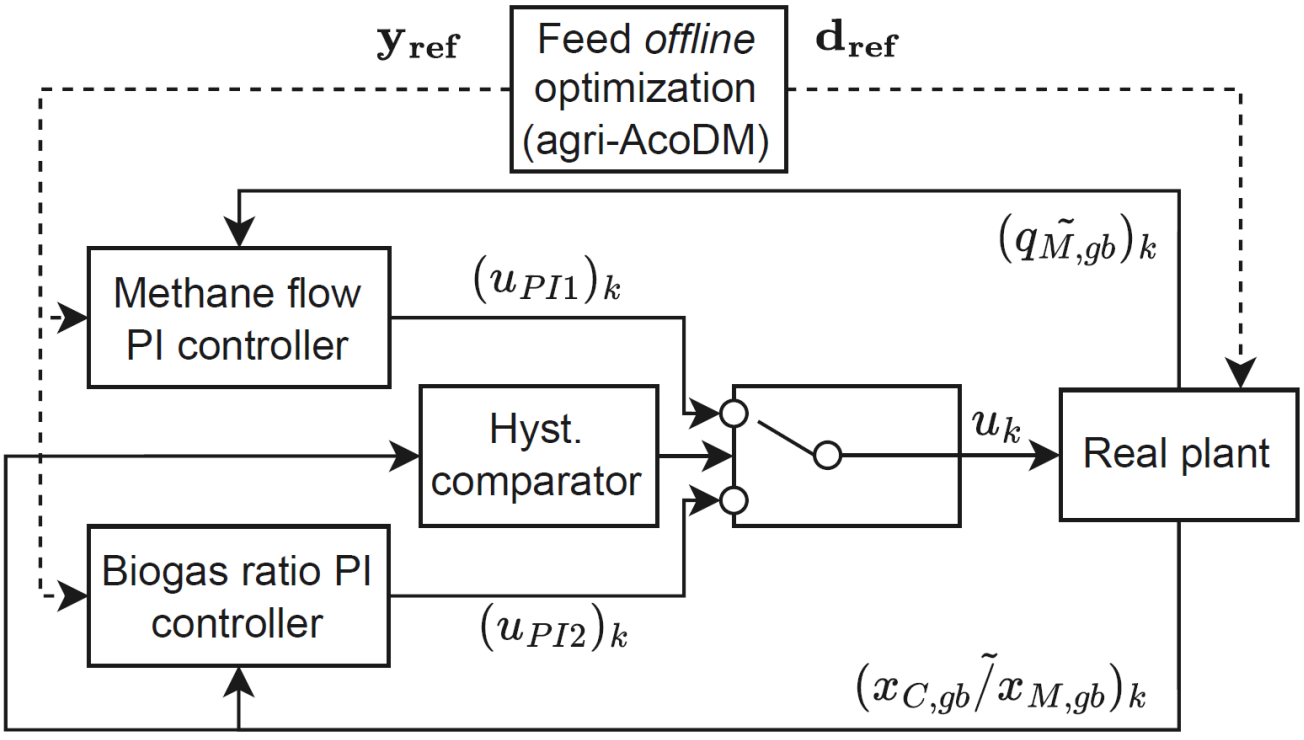
- The current control step *Timestamp* is defined as the current time (i.e. =*end_timestamp*) and has to be rounded to the nearest hour (HH:00:00) only if this program is coupled with **"cronjob.py"** (see note in the 'Closed-loop architecture' section of this manual). The user SETS THE NUMBER OF DAYS TO FILTER THE INPUT DATA CSV in the definition of *start_timestamp*.
- The user SPECIFIES THE DIRECTORY OF THE INPUT CSV DATA FILE that contains the *online* measurements (*Timestamp*, biogas rate '*gas_rate_out_ma*' (after outlier removal and moving average), biogas CH_4 '*xM_gb_out*' CO_2 '*xC_gb_out*' and fractions). In the example, *gas_rate_out_ma* must be in L h^{-1}, whereas *xM_gb_out* and *xC_gb_out* as fraction $\in [0, 1]$. The user SETS THE NUMBER OF DAYS TO FILTER THE INPUT DATA CSV in the definition of *start_timestamp*. CH_4 and CO_2 flow rates, and biogas CO_2/CH_4 ratio are computed. Data are extracted at the current control step *Timestamp* (last row of data).
- The user SPECIFIES THE DIRECTORY OF THE INPUT DATA FILE that contains the setpoints (Time or *Timestamp*, methane flow rate, biogas composition ratio and relative lower and higher thresholds). It can be in TXT or CSV format depending on how it is created. If TXT, as in the example, a dataframe must be created parsing Time to *Timestamp* values, specifying the *datetime* correspondent to the first Time value). For logging purposes, the user SPECIFIES THE DIRECTORY FOR AN OUTPUT CSV FILE where all the quantities of interest for the current control step are saved to.
- The SETTING OF THE CONTROL PARAMETERS, the **controllers' instantiation**, integrator state load, and the **override** and **compute** part are almost equal to what have been already described for "UIT_selectorPI_operative.py". Note: add the nominal/initial value of the control action to the *control_signal* to obtain the actual value of the control action to be communicated to the actuation system (see Section *Additional notes* for details).
- **Convert** the control action from a continuous *float* to a PWM on-off duty cycle couple (*tuple_seconds*). To do so, the user SPECIFIES THE ON-TIME SECONDS OF ONE WAVE PERIOD AND THE CHARACTERISTIC FLOW RATE OF THE PUMP. The user SPECIFIES THE DIRECTORY FOR AN OUTPUT FILE where the PWM signal is saved. The user SPECIFIES THE FILENAME AND DIRECTORY FOR AN OUTPUT CSV FILE where all the quantities of interest computed in the current control step are saved to.

- The **FlexiblePlotter class** is used to **plot** the main outputs of interest from *start_timestamp* to *now*. The user can select which data to plot and the plotting parameters of the **FlexiblePlotter** instantiation following the guidelines provided in its documentation (see...). The user SPECIFIES THE FILENAME AND DIRECTORY FOR THE OUTPUT PNG FIGURE.
- The last unindent code lines follow the **main()** declaration for it to be executed from the terminal (with command-line arguments) and other scripts (e.g. **"test_closedloop.ipynb"**).
  The values of the user-definable quantities/paths that are present in the current version inside the repository are the ones for this code to be ready for its actual implementation on a real system (coupled with the control architecture present in GitHub repo i.e. RPi- and relay-based). See the 'Closed-loop architecture' section for further details. See 'Closed-loop test' if, instead, the user desires to conduct closed-loop simulation tests.

Additional notes

If the input data file for setpoints is in TXT form, it means it is exactly the same file used as CombiTimeTable for the agri-AcoDM model (see...).
If the input data file for setpoints is in CSV form, it means it was created with ...
Generally, their primary source of generation is the **"Optimization_feed.ipynb"** code present inside the *'Biogoals.agri-AcoDM'* tool.

Closed-loop scheme



where k is the index of the current control step (in practice, the *Timestamp* in which the program is run).
Note that this control scheme was and shall be tuned around the *nominal/initial* equilibrium/operating point: the control action computed by the **PIcontroller** class shall be

null when working around such point. The control action required to sustain such working point (i.e. the *nominal_u*) must be then added to the *control_signal* returned by the *compute* method of the **PIcontroller** class. In the scheme above, u_{PI1} is the control action's correction/deviation computed by the *header*, whereas u_{PI2} is the one computed by the *follower*. One of the two is selected based on the boolean information provided by the **HysteresisComparator** block, and finally the nominal/initial value of the control action (u_0 in **"SelectorPI_pseudo_code.pdf"**) is added to obtain the actual value to be given to the PWM converter function.

Closed-loop architecture

To use ***"RPi_selectorPI_operative.py"*** on a real reactor, the user shall couple it with the the codes present in this GitHub repo, that contains the "practical architecture" implementation of the closed-loop i.e. code to communicate with sensor, preprocessing data and give the control signal to actuator (e.g. **"preprocess_meas.py"**, **"cronjob.py"**, **"actuator_set.py"** etc.). Just copy them locally (download with *git clone*) running in a terminal:

```
cd path\to\somewhere
git clone https://github.com/DaveCacci/B.Architecture.git
```

Then, inside the **"RPi_selectorPI_operative.py"** add to the imports:

```
import sys
sys.path.append(r"path\to\somewhere\B.Architecture")
from filename_py import function_name
```

(even easier if *git clone* is run in the same directory of **"RPi_selectorPI_operative.py"**, no need to `sys.path.append`).

Design optimisation

The **"Optimization_selectorPI_design.ipynb"** notebook must be run once a closed-loop model in Modelica that simulates the behaviour of the selectorPI controller over the agri-AcoDM model used as real plant is set up with a "nominal"/"initial" design of the controller parameters (mainly the k_P and T_I values). Then, the **"modelica_optimizer.py"** and **"modelica_integrator.py"** are used to:

- Run the Modelica "nominal" simulation $\rightarrow$ extract "nominal" quantities obtained with the "nominal" k_P and T_I values (optimisation variables).
- Declare cost function type (e.g. RMSE), and parameter and output constraints (e.g. feasible set of parameter values and upper bounds for total VFA and COD content in digestate).
- Run direct-search optimisation (slow but global methods, such as "nelder-mead" and "differential evolution") $\rightarrow$ obtain the "optimal" values of the k_P and T_I, the best value of the cost function.
- Run the Modelica "optimal" simulation $\rightarrow$ extract "optimal" quantities obtained with the "optimal" k_P and T_I values found above.

- Compare "nominal" with "optimal" quantities.
  Use the "optimal" values found above to conduct further simulation tests to assess robustness and, if satisfactory, proceed with the experimental validation setting these values inside **"UIT_selectorPI_operative.py"** or **"RPi_selectorPI_operative.py"**.

Closed-loop test

The **"test_closedloop.ipynb"** Jupyter notebook allows the user to conduct many different simulation study of the closed-loop behaviour of the selectorPI controller. It calls two "main" functions i.e. the **"RPi_selectorPI_operative.py"** and **"test_closedloop_integrator.py"** for feedback control and integration (simulator of the real plant) purposes respectively. The **"SelectorPI_pseudo_code.pdf"** tries to elucidate the main logical and mathematical causality that makes the loop functioning correctly.
To run a closed-loop test, the user needs:

- to install the *Biogoals.agri-AcoDM* library or have a working Modelica model that is consistent with the settings present in the **"test_closedloop_integrator.py"** example. This is used to simulate the real plant;
- to install the *Biogoals.Twin* library or at least download and place in the **"test_closedloop_integrator.py"** directory the two python files present in such library and called **model_inputs.py** and **model_read_file.py**.
  Add the path to *sys* if these files are placed somewhere else locally in the user machine.
  For consistency, the *'modelname'* specified as the first argument of *main_selector* in **"test_closedloop.ipynb"** must be the same set in **"test_closedloop_integrator.py"**.
  Same holds for '*testname*' inside **"RPi_selectorPI_operative.py"** that must be equal to the one specified inside **"test_closedloop_integrator.py"**.
  To run the closed-loop test consistent with the example files provided, the user must change some settings inside the current implementation of **"RPi_selectorPI_operative.py"**:
- Change the path and filename of *'y_df_on_path'* to be equal to the one specified inside **"test_closedloop_integrator.py"** (i.e. '*{modelname}_y_df_on.txt'*).
- Change the name of the outputs (columns of *'y_df_data_on'*, '*mean*', '*stdev*', *'rate'*, *'ch4'*, *'co2'*) read to be consistent to the ones specified inside **"test_closedloop_integrator.py"** (i.e. '*y_df_adm1.columns*').
  The settings that the user can modify inside **"test_closedloop_integrator.py"** are, primarily:
- the path where the **model_inputs.py** and **model_read_file.py** functions are located;
- the path where the Modelica model to be integrates is placed ('*path_to_agriacodm*');
- meta-options such as *'modelname'* and *'testname'*. Name of the sub-folder where the logs will be placed, and relative filename;
- inputs to the "**read_all**" and "**prepare_model_inputs**" functions (see their documentation), for example the name of the file used to store the Modelica model's state values for recursive intialization ('*modelica_states_storage_filename'*);

- whether additional time-varying disturbances are added to the Modelica model (e.g. *'fake_theta_reduction'*, see [add IEEE_CDC2025 link](insert here)).
- inputs to the "**modelica_integrator**" function (see its documentation);
- the names of the columns that will be used to save the outputs extracted from Modelica (e.g. *'gas_rate'*, *'xch4_interp'*, *'xco2_interp'*);
- the level of noise added to the integrator outputs before saving them as measurements (i.e. the *'cov'* of the '*np.random.multivariate_normal*' function).
  Note: in the Modelica model be sure that the CombiTimeTables generate by this code are the ones actually read!
  Note: reactor's working temperature, volume and pressure are stored **"parameters_update.json"** (see below) and eventually needed to convert the biogas flows from L h^{-1} to mmol L^{-1} d^{-1} (normal conditions) and viceversa.

In the example provided, the *testname* = 'Test_closedloop'. Inside this sub-folder a very similar structure of the ***"RPi_implementation/"*** folder is present to store input and outputs. The content of this sub-folder is briefly described hereafter:

- ***"Test_closedloop/1"***: sub-folder to collect all the input/output files read and generated by the OpenModelica Compiler (OMC) at runtime. (e.g. the MOS file that sets the simulation options and parameters, the LOG file of the simulation and the MAT file that stores the results). In this example is called *"Test_closedloop/1*" since it was set *modelname* = '1'.
- ***"Test_closedloop/Input/"***: sub-folder that contains the inputs read by the main code functions. Note that "input" is intended both in the sense of the "code" and of the "control scheme" (even tough the last sense is actually an "output" of the main code functions):
  - ***"1_u_actual.csv"***: CSV file storing the (actual) control action computed by the controller along with the Timestamp of control run (in mL d^{-1}).
  - ***"fake_theta_reduction.csv"***: CSV file containing the trajectory of the additional time-varying disturbance inside the agri-AcoDM (artificial $k_{m,ac}$ reduction in this example, see [add IEEE_CDC2025 link](insert here)).
  - ***"Manual_flowrates_real_R1.csv"***: CSV file containing the flow rates of the non-controllable co-feedstocks (called d_{ref} in the pseudo-code and in the closed-loop scheme of section 'Closed-loop scheme').
  - ***"Output_setpoints.txt"***: TXT input data file containing the setpoints.
  - ***"SELECTOR_pwm_actual.csv"***: CSV file storing the (actual) control action computed by the controller along with the Timestamp of control run, but in terms of on/off times for actuator's activation. To be read by the "actuator_set.py" code inside the "practical architecture" library (see section 'Closed-loop architecture').
  - ***"States_agriAcoDM_real.csv"***: CSV file storing the state vector to initialise the agri-AcoDM model at each control run (called $\mathbf{x}$ in the pseudo-code).
  - ***"CowSlurry_comp_raw.csv", "MizeSilage_comp_raw.csv", "TomatoSauce_comp_raw.csv"***: for the example of interest, the CSV files containing the composition of the co-feedstocks, as read also by the agri-AcoDM

model and as given by the Feedstocks_composition.xlsx Excel file present inside the *Biogoals.agri-AcoDM* repository (see its documentation for further details).

- ***"Test_closedloop/logs/"***: sub-folder that saves a copy of the logging massages shown at runtime in console, for later inspection. Note: saves all messages returned by the *logging* Python-library, not the simpler messages returned using the *print* Python-keyword.
- ***"Test_closedloop/Output/"***: sub-folder that contains all the different CSV files outputted by the main control program, equal to the ones already described in the 'Files description' section.
- ***"Test_closedloop/plots/"***: sub-folder to store the plots created by the main control program.
- ***"Test_closedloop/plots_postprocess/"***: generate by "test_closedloop_integrator.py" after the completion of a closed-loop simulation, in order to save figures and store results for further analyses.
- ***"Test_closedloop/1_y_df_off.txt"***: TXT file that stores the non-(continuously)-measurable outputs from the agri-AcoDM integration. Used only for post-process analysis and plotting purposes.
- ***"Test_closedloop/1_y_df_on.txt"***: TXT file that stores the measurable outputs from the agri-AcoDM integration (called $\tilde{\mathbf{y}}$ in the pseudo-code i.e. $q_{\tilde{M},gb}$ and $x_{C,gb}/\tilde{x}_{M,gb}$ in the 'Closed-loop scheme' section) and read by the selectorPI controller.
- ***"Test_closedloop/integrator.json"***: JSON that contains the simulation options i.e. integration options such as start and end time, integration time step and tolerance (called $\boldsymbol{\theta}_{\text{integrator}}$ in the pseudo-code).
- ***"Test_closedloop/parameters_update.json"***: JSON that contains some process parameters (e.g. used in this program for the conversion of the biogas flow rate unit of measurement).